Basic Exercises for Competitive Programming - Python

Jan Pol

The following book shows a compilation of more than 20 basic exercises for competitive programming, all of them are written in Python. In addition, desktop tests are added to observe the operation of each algorithm.

Hoping you can take knowledge of the pages of this book, my best wishes.

Jan Pol

Exercise 1

Write a program that reads a string S, and prints that line on the standard output in reverse, that is, flipped right-to-left.

Input

The only input has a string S.

Output

Print the string in reverse

Example

Input

1 2 3 hello

Output

olleh 3 2 1

Solution

One way to easily solve this task is to use a loop to print the characters from the last position to the first. This avoids the need to save the reverse list, just print each character.

The code for the solution written in Python is shown below.

```
1 l = input()
2 i = len(l)
3 while i > 0:
4     i = i - 1
5     print(l[i], end='')
6 print()
```

Desktop Testing

Doing a desktop testing, it is observed how in each iteration each character is added inversely.

l	i	print
1 2 3 hello	11	
	10	o
	9	ol
	8	oll
	7	olle
	6	olleh
	5	olleh
	4	olleh 3
	3	olleh 3
	2	olleh 3 2
	1	olleh 3 2
	1	olleh 3 2 1

Exercise 2

Given of input a positive integer n. If n is even, the algorithm divides it by two, and if n is odd, the algorithm multiplies it by three and adds one. The algorithm repeats this, until n is one. For example, the sequence for n=5 is as follows:

5→16→8→4→21

Your task is to simulate the execution of the algorithm for a given value of n.

Input

The only input line contains an positive integer n.

Output

Print a line that contains all values of n separated with spaces.

Example

Input

5

Output

5 16 8 4 21

Solution

To solve this task, a cycle is used to verify that the number is not yet 1. Within the cycle it is checked if the number is even, it is divided by two, otherwise the number is multiplied by 3 and 1 is added to it. the number is printed, like this until the number is 1.

The code for the solution written in Python is shown below.

```
1 n = int(input())
2 print(n, end= ' ')
3 while n > 1:
4    if n % 2 == 0:
5       n //= 2
6    else:
7       n = 3 * n + 1
8    print(n, end=' ')
```

Desktop Testing

Doing a desktop testing, it shows if the number is odd or even and the operation is performed.

Operation	n	Even or odd
	5	odd
3 * n + 1	16	odd
3 * n + 1	8	even
n // 2	4	even
n // 2	2	even
n // 2	1	

Exercise 3

Write a program that reads a list of non-negative numbers from the standard input, with one number per line, and prints a histogram corresponding to those numbers. In this histogram, each number N is represented by a line of N characters "#"

Input

The only line contains n non-negative integers.

Output

Print the histogram result.

Example

Input

10 15 7 9 1 3

Output

```
##########
###############
#######
#########
#
###
```

Solution

To solve this task, it is taken into account that a list of non-negative integers will be received so that they are saved using a “map” of integers for the input, removing the spaces. Then '#' is printed as many times as the value of the number.

The code for the solution written in Python is shown below.

```
1 arr = map(int, input().split())
2
3 for a in arr:
4  print('#' * a)
```

Desktop Testing

Doing a desktop testing, it is shown how in the next line of each iteration as many '#' as the value of the number are added.

arr	a	print
10 15 7 9 1 3	10	##########
	15	########## ###############
	7	########## ############### #######
	9	########## ############### ####### #########
	1	########## ############### ####### ######### #
	3	########## ############### ####### ######### # ###

Exercise 4

You are given all numbers between 1,2,…,n except one. Your task is to find the missing number.

Input

The first input line contains an integer n.

The second line contains n−1 numbers. Each number is distinct and between 1 and n (inclusive).

Output

Print the missing number.

Example

Input

10

2 8 10 6 5 1 3 7 4

Output

9

Solution

To solve this task, the integer n is saved. Then the integer line is saved. A sum is initialized, each integer is added. At the end, the Gauss sum formula is used: n * (n + 1) / 2. The previously made sum is subtracte it to obtain the missing number. This works because the Gauss sum gets the real total sum based on the total of numbers, if the sum of the provided numbers is subtracted, the result will show the missing value to get the real total sum.

The code for the solution written in Python is shown below.

```
1 n = int(input())
2 arr = map(int, input().split())
3 s = 0
4
5 for a in arr:
6     s += a
7
8 print(n * (n + 1) // 2 - s)
```

Desktop Testing

Doing a desktop testing, it is shown how in each iteration the respective number is added. At the end the operation is printed.

n	s	a
10	0	
	0	2
	2	8
	10	10
	20	6
	26	5
	31	1
	32	3
	35	7
	42	4
	46	

The final result is:

10 * (10 + 1) // 2 – 46

10 * 11 // 2 – 46

110 // 2 – 46

55 – 46

Result: 9

Exercise 5

Given a sequence of integers A, a maximal identical sequence is a sequence of adjacent identical values of maximal length k. Write a program that reads an input sequence A, with one or more numbers per line, and outputs the value of the first (leftmost) maximal identical sequence in A.

Input

The first input line contains an integer n.

The second line contains n integers.

Output

Print one integer: the length of the longest repetition

Example

Input

2 4 6 2 2

Output

2

Solution

To solve this task, an efficient way to know which number is the most repeated is to use variables that help us in the process. In this case, 'k' is used to determine how much a number is repeated, 'i' saves the previous index, 'j' saves the current index and 'v' stores the number that is repeated the most sequentially.

At the beginning of the cycle, the previous and current values are compared, if they are different, the indices are exchanged. The index 'j' increases and is compared if the numbers that are repeated is greater than 'k', if so, 'v' takes the value of the number and 'k' takes the value of how much that number was repeated.

The code for the solution written in Python is shown below.

```
A = list(map(int, input().split()))

v = A[0]
k = 1
i = 0
j = 1
while j < len(A):
  if A[j] != A[i]:
     i = j
  j += 1
  if j - i > k:
     v = A[i]
     k = j - i

print(v)
```

Desktop Testing

Doing a desktop testing, it is observed how in each cycle the previous number and the current number are compared, as well as the increase or exchange in the indexes according to whether the sequential number is different. When the numbers compared are equal, the index 'j' increases more than the 'i', when not, the indexes go hand in hand.

Below is the sequence of integers 'A' and the desk test.

A

2	4	6	2	2

v	A[j]	A[i]	k	i	j
2			1	0	1
	4	2		1	0
					1
	4	4			2
	6	4		2	1
					2
	6	6			3
	2	6		3	2
					3
	2	2			4
	2	2			5
2			2		

Exercise 6

You are given a DNA sequence: a string consisting of characters A, C, G, and T. Your task is to find the longest repetition in the sequence. This is a maximum-length substring containing only one type of character.

Input

The only input line contains a string of n characters.

Output

Print one integer: the length of the longest repetition.

Example

Input

ATTGCCCA

Output

3

Solution

To solve this task, the input string is taken, using the variables 'ans' stores the response, 'count' counts the repetitions and 'l' contains the character to compare. A loop is used to go through each character in the string, if the character is repeated, add the counter and replace 'ans' with the maximum between 'count' and 'ans', otherwise 'l' becomes the new character and the counter is reset.

The code for the solution written in Python is shown below.

```
s = input()
ans = 1
count = 0
l = 'A'
for cha in s:
    if cha == l:
        count += 1
        if count > ans:
            ans = count
    else:
        l = cha
        count = 1
print(ans, end=' ')
```

Desktop Testing

Doing a desktop testing, it shows how each character of the sequence is compared, each that is equal increases the counter and the maximum between the response is verified, when the values are not reset.

```
s = ATTCGGGA
```

cha	count	l	ans
	0	A	1
A	1		
T	1	T	
T	2		2
G	1	G	
C	1	C	
C	2		
C	3		3
A	1	A	

Exercise 7

Write a program that takes a number N (a positive integer) in decimal notation from the console, and prints its value as a roman numeral.

Input

The only input line has an integer n.

Output

Print the value as a roman numeral.

Example

Input

2021

Output

MMXXI

Solution

To solve this task, the predefined characters of the Roman numerals are listed, separated by groups, from one to nine, from ten to 90 and from 100 to 900, each one with the number zero. It is checked if the quantity is greater than or equal to 1000, if so, the symbol 'M' is added as many times as thousands it contains. Then the hundreds are extracted and replaced by their respective symbols, in the same way the tens and units, printing the result at the end.

The code for the solution written in Python is shown below.

```
 1 ten_zero = ['', 'I', 'II', 'III', 'IV', 'V', 'VI', 'VII',
'VIII', 'IX']
 2 ten_one = ['', 'X', 'XX', 'XXX', 'XL', 'L', 'LX', 'LXX',
'LXXX', 'XC']
 3 ten_two = ['', 'C', 'CC', 'CCC', 'CD', 'D', 'DC', 'DCC',
'DCCC', 'CM']
 4
 5 n = int(input())
 6
 7 r = ''
 8 while n >= 1000:
 9     r += 'M'
10     n -= 1000
11
12 c = n // 100
13 r += ten_two[c]
14 n -= c*100
15
16 d = n // 10
17 r += ten_one[d]
18 n -= d*10
19
20 r += ten_zero[n]
21
22 print(r)
```

Desktop Testing

Doing a desktop testing, it is shown how each thousand is subtracted from the quantity and the symbol 'M' is added, in the same way the hundreds, tens and units are extracted to obtain the result.

r	n	c	d
‘’	2021		
‘M’	1021		
‘MM’	21	0	2
‘MMXX’	1		
‘MMXXI’			

Exercise 8

You are given an array of n integers. You want to modify the array so that it is increasing. Every element is at least as large as the previous element.

On each move, you may increase the value of any element by one. Your task is find the minimum number of moves required.

Input

The first input line contains an integer n, the size of the array.

Then, the second line contains n integers, the contents of the array.

Output

Print the minimum number of moves.

Example

Input

5

2 1 4 1 5

Output

4

Solution

To solve this task, we must find the least amount of movements necessary to increase the values of an array so that the list increases. After reading the input data, the variables 'mx' are initialized, the maximum and 'ans' are stored with a value of zero, each one of the data in the array is traversed, the maximum is searched and the value of the number is subtracted, the result is saved in response. This works because it is looking for if the number before the current one is greater, it obtains the difference between current and previous, in other words it obtains the increase that is needed so that both values are equal.

The code for the solution written in Python is shown below.

```
n = int(input())
arr = map(int, input().split())
mx = 0
ans = 0

for x in arr:
    mx = max(x, mx)
    ans += mx - x

print(ans)
```

Desktop Testing

Doing a desktop testing, it is shown how the maximum value is obtained, at the moment in which the value of x decreases, it obtains the necessary value to be equal to the previous one.

```
arr = 2 1 4 1 5
```

X	mx	ans
	0	0
2	2	0
1		1
4	4	1
1		4
5	5	4

Exercise 9

A ‘beautiful’ permutation is a permutation of integers 1,2,…,n, if there are no adjacent elements whose difference is 1.

Given n, construct a beautiful permutation if such a permutation exists. If there are no solutions, print “NO SOLUTION”

Input

The only input line contains an integer n.

Output

Print a beautiful permutation of integers 1,2,…,n. If there are several solutions, you may print any of them. If there are no solutions, print "NO SOLUTION".

Example

Input

5

Output

2 4 1 3 5

Solution

To solve this task, the difference between elements is required to be different than 1. When n is equal to 1, 1 is printed, if it is less than 4 there is no solution, in all other cases, it will be enough to print 2 by 2 starting at 2 up to the value of n + 1 and in another cycle the same strategy but starting at 1.

The code for the solution written in Python is shown below.

```
n = int(input())
if n == 1:
 print(1, end=' ')
elif n < 4:
 print('NO SOLUTION')

else:
 for i in range(2, n+1, 2):
  print(i, end=' ')

 for i in range(1, n+1, 2):
  print(i, end=' ')
```

Desktop Testing

Doing a desktop testing, it is easy to observe that it is only enough to print numbers from 2 to 2 with different start, in this way it ensures that the difference between the numbers is not 1.

n	print
5	2
	2 4
	2 4 1
	2 4 1 3
	2 4 1 3 5

Exercise 10

A number spiral is an infinite grid whose upper-left square has number 1. Here are the first five layers of the spiral:

1	2	9	10	25
4	3	8	11	24
5	6	7	12	23
16	15	14	13	22
17	18	19	20	21

Your task is to find out the number in row x and column y.

Input

The first input line contains an integer t, the number of tests.

After this, there are t lines, each containing integers x and y.

Output

For each test, print the number in row x and column y.

Example

Input

2

3 4

4 1

Output

12

16

Solution

To solve this task it is not necessary to make a matrix that contains the infinite grid, first the input value is stored, then for each 'x', 'y' the highest is obtained and it is saved in 'z', in 'z2' the multiplication between the previous number to the greater is saved by itself. If ‘z’ is odd, and ‘x’ is the highest, it will suffice to add ‘y’ to ‘z2’, otherwise double ‘z’ minus x is added to ‘z2’. If ‘z’ is even, and ‘y’ is the highest, just add ‘x’ to ‘z2’, otherwise double ‘z’ minus ‘y’ is added to ‘z2’.

The code for the solution written in Python is shown below.

```
t = int(input())
for i in range(t):
    x, y = map(int, input().split())
    z = max(x, y)
    z2 = (z - 1) * (z - 1)
    if z % 2 != 0:
        if x == z:
            ans = z2 + y
        else:
            ans = z2 + 2 * z - x
    else:
        if y == z:
            ans = z2 + x
        else:
            ans = z2 + 2 * z - y
    print(ans)
```

Desktop Testing

Doing a desktop testing, it can be observed how, being even and if 'z' and 'y' are the same, different operations are carried out but always starting from the same base which is 'z2'.

t	x	y	z	z2	ans
2	3	4	4	3*3 = 9	9 + 3 = 12
	4	1	4	3*3 = 9	9 + 2 * 4 – 1 = 16

Exercise 11

Write a program that reads nine lines from the input each containing nine numbers between 1 and 9, representing a sudoku puzzle. The program should output "Correct" if the input represents a valid sudoku or "Incorrect" in otherwise.

Input

Nine lines, each containing nine numbers between 1 and 9.

Output

Print "Correct" if the input represents a valid sudoku or "Incorrect" in otherwise.

Example

Input

1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 1 2 3
7 8 9 1 2 3 4 5 6
2 3 4 5 6 7 8 9 1
5 6 7 8 9 1 2 3 4
8 9 1 2 3 4 5 6 7
3 4 5 6 7 8 9 1 2
6 7 8 9 1 2 3 4 5
9 1 2 3 4 5 6 7 8

Output

Correct

Solution

To solve this task, a method is made to verify if the permutation is correct, if there are no repeated numbers and they are in the range from 1 to 9. Another method is used for the addition of matrices. When entering the sudoku game, it goes through 3 by 3 verifying each permutation. If the permutation is incorrect, it prints and the program ends, otherwise it prints that it is correct.

The code for the solution written in Python is shown below.

```
def perm9(V):
    F = [ 0, 0, 0, 0, 0, 0, 0, 0, 0 ]
    for x in V:
        if x < 1 or x > 9 or F[x-1] != 0:
            return False
        else:
            F[x-1] = 1
    for x in F:
        if x == 0:
            return False
    return True

def submatrix(S, x, y):
    return S[x][y:y+3] + S[x+1][y:y+3] + S[x+2][y:y+3]

S = [list(map(int, input().split())) for _ in range(9)]

for i in range(0,9,3):
    for j in range(0,9,3):
        if not perm9(submatrix(S,i,j)):
            print("Incorrect")
            sys.exit(1)

print("Correct")
```

Desktop Testing

Doing a desktop testing, it shows how the numbers are stored in a range of 3 by 3, they go on to check if the permutation is correct. Since they are all correct, “Correct” is printed.

```
S =
1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 1 2 3
7 8 9 1 2 3 4 5 6
2 3 4 5 6 7 8 9 1
5 6 7 8 9 1 2 3 4
8 9 1 2 3 4 5 6 7
3 4 5 6 7 8 9 1 2
6 7 8 9 1 2 3 4 5
9 1 2 3 4 5 6 7 8
```

i	j	submatrix	perm9
0	0	[1, 2, 3, 4, 5, 6, 7, 8, 9]	True
0	3	[4, 5, 6, 7, 8, 9, 1, 2, 3]	True
0	6	[7, 8, 9, 1, 2, 3, 4, 5, 6]	True
3	0	[2, 3, 4, 5, 6, 7, 8, 9, 1]	True
3	3	[5, 6, 7, 8, 9, 1, 2, 3, 4]	True
3	6	[8, 9, 1, 2, 3, 4, 5, 6, 7]	True
6	0	[3, 4, 5, 6, 7, 8, 9, 1, 2]	True
6	3	[6, 7, 8, 9, 1, 2, 3, 4, 5]	True
6	6	[9, 1, 2, 3, 4, 5, 6, 7, 8]	True

Exercise 12

Your task is to count for k=1,2,…,n the number of ways two knights can be placed on a k×k chessboard so that they do not attack each other.

Input

The only input line contains an integer n.

Output

Print n integers, the results.

Example

Input

5

Output

0
6
28
96
252

Solution

To solve this task, to obtain each one of the ways in which they can be displaced if they are attacked first, the integer 'n' is stored. For each 'k' (length of the matrix) up to 'n + 1' starting at 1, 'a1' stores the total number of cells on the board, 'a2' stores the total amount in which they can move around the board. At the time the board is 3 or more, if two knights attack each other, they will be in a 2 * 3 rectangle or a 3 * 2 rectangle. So the number of ways to place them is (n-1) * (n-2) + (n-2) * (n-1). The total ways to position the knight to attack each other will be 2 * 2 * (n-1) * (n-2) = 4 * (n-1) * (n-2). This operation must be subtracted from the total movements of both knights, you get the amount of movements they can make without attacking.

The code for the solution written in Python is shown below.

```
n = int(input())
for k in range(1, n + 1):
    a1 = k * k
    a2 = a1 * (a1 - 1) // 2
    if k > 2:
        a2 -= 4 * (k - 1) * (k - 2)
    print(a2)
```

Desktop Testing

Doing a desktop testing, it is shown that when ‘k’ is greater than 2 it begins to subtract the positions of the knights in which they can attack. When 'k' is worth 1 and 2 it is not subtracted because they will never meet to attack each other.

```
n = 5
```

k	a1	a2
1	1	0
2	4	6
3	9	36
		28
4	16	120
		96
2	25	300
		252

Exercise 13

Your task is to divide the numbers 1,2,…,n into two sets of equal sum.

Input

The only input line contains an integer n.

Output

Print "YES", if the division is possible, and "NO" otherwise.

If the division is possible. Print the number of elements in the first set followed by the elements themselves in a separate line, and then, print the second set in a similar way.

Example

Input

3

Output

YES

1

3

2

2 1

Solution

To solve this task, it first checks if the Gauss sum of the number is even, if it is, it means that it cannot be divided into two sets of sums. Otherwise, two lists and half the Gauss sum are created. For each number in descending order, if the sum is greater than the number, it is subtracted from the sum and stored in one list, otherwise it is stored in the other.

The code for the solution written in Python shown below.

```
n = int(input())
if n * (n + 1) / 2 % 2:
   print("NO")
else:
   print("YES")
   v1 = []
   v2 = []
   sum = n * (n + 1) / 4
   for i in range(n, 0, -1):
       if sum >= i:
          sum -= i
          v1.append(i)
       else:
          v2.append(i)

   print(len(v1))
   print(*v1)
   print(len(v2))
   print(*v2)
```

Desktop Testing

Doing a desktop testing, it shows that the Gauss sum is not even. It is also observed that when 'i' is greater than or equal to the sum, it is stored in 'v1' and is subtracted, for the other numbers none is greater than or equal to the sum, so they are stored in 'v2'.

```
n = 3
YES
```

i	sum	v1	v2
	3	[]	[]
3	0	3	
2			2
1			2 1

Exercise 14

Your task is to calculate the number of bit strings of length n.

For example, if n=2, the correct answer is 4, because the possible bit strings are 00, 01, 10 and 11.

Input

The only input line has an integer nn.

Output

Print the result modulo 109+7.

Example

Input

7

Output

128

Solution

To solve this task, it is very simple, just multiply the base 2 to the power n, the task specifies that the result must be printed in module 109+7, so the module of the result is obtained.

The code for the solution written in Python is shown below.

```
1 n = int(input())
2 print(2 ** n % (10 ** 9 + 7))
```

Desktop Testing

Doing a desktop testing, the ease of the algorithm is shown, it executes a simple operation.

n	print
7	2 ** 7 % (10 ** 9 + 7) = 128

Exercise 15

Write a program that reads a list of numbers from the standard input, with one or more numbers per line, and prints a vertical histogram corresponding to those numbers. In this histogram, each number N is represented by a column of N characters "#" starting from a base-line of N dash characters "-" representing value 0. Positive numbers are represented by a column above zero while negative numbers are represented with a column below zero.

Input

The only input line contains n integers.

Output

Print the vertical histogram.

Example

Input

2 -1 3 1

Output

```
  .#
# #
# ##
----
 #
```

Solution

To solve this task, a method is created to which a list will be passed, the variables 'top' the highest stack of '#' and 'bottom' the lowest stack are used. A cycle is used to print each of the '#' corresponding to its positive numbers. Then the delimiter '-' is printed. At the end, using another cycle, as many '#' as there are numbers are printed.

The code for the solution written in Python is shown below.

```
def v_histogram(A):
    top = 0
    bottom = 0
    for a in A:
        if a > top:
            top = a
        elif a < bottom:
            bottom = a

    i = top
    while i > 0:
        for a in A:
            if a >= i:
                print('#',end='')
            else:
                print(' ',end='')
        print()
        i = i - 1

    for a in A:
        print('-',end='')

    print()
    while i > bottom:
        i = i - 1
        for a in A:
            if a <= i:
                print('#',end='')
            else:
                print(' ',end='')
```

```
31         print()
32
33 n = list(map(int, input().split()))
34 v_histogram(n)
```

Desktop Testing

Doing a desktop testing, it shows how '#' is printed according to the top, if the number is greater than 'i' it is printed '#' otherwise ' ' is printed to space. Then the middle part is printed. In the end, the negative part is printed similarly to the positive part.

A = 2 -1 3 1

a	top	bottom	i	print
	0	0		
Find top an bottom				
2	2			
-1		-1		
3	3			
1				
Print positive numbers				
			3	
2				
-1				
3				#
1				#
2			2	#
				#
-1				#
				#

3				
1				
2			1	
-1				
3				
1				
Print the middle				
			0	

Print negative numbers				
2			-1	
-1				
3				
1				

Exercise 16

Given an integer n, calculate the number of trailing zeros in the factorial n!.

Input

The only input line has an integer n.

Output

Print the number of trailing zeros in n!.

Example

Input

10

Output

2

(10! = 3628800)

Solution

To solve this task, a mathematical formula of the form: n / 5 + n / 25 + n / 125 is used. Therefore, the variable 'ans' is initialized in 0 and 'i' in 5. A cycle is used, for each iteration the variable 'ans' is added 'n' divided by 'i', the value of 'i' it is multiplied by 5 and at the end of the cycle the answer is printed.

The code for the solution written in Python is shown below.

```
n = int(input())
ans = 0
i = 5
while i <= n:
    ans += n // i
    i *= 5

print(ans)
```

Desktop Testing

When doing a desktop test, it shows that it only enters the cycle once. You get the result instantly.

n	ans	i
10	0	5
	2	25

Exercise 17

Given two coin piles containing a and b coins. On each move, you can either remove one coin from the right pile and two coins from the left pile, or two coins from the right pile and one coin from the left pile.

Your task is to efficiently find out if you can empty both the piles.

Input

The first input line has an integer t, the number of tests.

t lines, each of which has two integers a and b, the numbers of coins in the piles.

Output

For each test, print "YES" if you can empty the piles and "NO" otherwise.

Example

Input

2

2 4

4 4

Output

YES

NO

Solution

To solve this task, for each pair of values received, three conditions are passed, the first condition ensures that the sum of both is a mod of 3 and the remaining two conditions ensure that the coins can be taken in the proper order.

The code to solve this task written in Python is shown below.

```
1 t = int(input())
2 for i in range(t):
3     a, b = map(int, input().split())
4     print("YES" if(a + b) % 3 == 0 and 2 * a >= b and 2 * b
>= a else "NO")
```

Desktop Testing

Doing a desktop testing, it is shown that in the first case the three conditions are fulfilled, therefore 'YES' is printed. In the other case the condition of mod 3 is false, therefore 'NO' is printed.

t	a	b	conditional	print
2	2	4	if 6 % 3 == 0 and 4 >= 4 and 8 >= 2	YES
	4	4	if 8 % 3 == 0 and 8 >= 4 and 8 >= 4	NO

Exercise 18

Given a string, your task is to reorder its letters in such a way that it becomes a palindrome, it reads the same forwards and backwards.

Input

The only input line has a string of length n consisting of characters A–Z.

Output

Print a palindrome consisting of the characters of the original string. You may print any valid solution. If there are no solutions, print "NO SOLUTION".

Example

Input

ababaccba

Output

Aacbbbcaa

Solution

To solve this task, the ‘collections’ library is used to convert the string into a Counter. The variable ‘chance’ is used to add the number of odd letters, ‘fail’ when there is no solution, ‘arr’ to save the letters of even numbers and ‘mid’ for the odd ones. For each letter in the Counter, if the amount is even, it is saved in 'arr', otherwise depending on the value of 'chance', the odd value is saved in 'mid' or it has no solution. The last is joined by ‘arr’ plus ‘mid’ and the inverse of ‘arr’.

The code for the solution written in Python is shown below.

```
import collections
S = str(input())
D = collections.Counter(S)
chance = True
fail = False
arr = []
mid = []
for c in D:
   if D[c] & 1 == 0:
      arr.append(c*(D[c]//2))
   else:
      if chance:
         mid.append(c*(D[c]))
         chance = False
      else:
         print('NO SOLUTION')
         fail = True
         break
if not fail:
   print(''.join(arr) + ''.join(mid) + ''.join(arr[::-1]))
```

Desktop Testing

Doing a desktop testing, it is observed if in 'D [c]' it is odd it is stored in 'mid', on the other hand it is stored in 'arr'. At the end the print is shown joining the parts.

```
S = abacbca

D = {'a': 3, 'b': 2, 'c': 2}
```

c	D[c]	arr	mid	chance	fail
		[]	[]	True	False
a	3		aaa	False	
b	2	b			
c	2	bc			

The final result is:

```
print('bc' + 'aaa' + 'cb')

>> bcaaacb
```

Exercise 19

Given a string, your task is to generate all different strings that can be created using its characters.

Input

The only input line has a string of length n. Each character is between a–z.

Output

First print an integer k: the number of strings.

Then print k lines: the strings in alphabetical order.

Example

Input

abc

Output

6
abc
acb
bac
bca
cab
cba

Solution

To solve this task, the 'permutations' package is imported, the permutations of the entered string are saved. The number of chains is printed, at the end each of the chains are printed.

The code for the solution written in Python is shown below.

```
1 from itertools import permutations
2 s = input()
3 ar = [''.join(e) for e in permutations(s)]
4
5 print(len(ar))
6 for e in ar:
7    print(e)
```

Desktop Testing

Doing a desktop testing, it is shown that after printing the length, in a simple way the arrangement is only traversed and each permutation is printed.

S = abc

len(arr)	e	print
6	abc	abc
	acb	abc acb
	bac	abc acb bac
	bca	abc acb bac bca
	cab	abc acb bac bca cab
	cba	abc acb bac bca cab cba

Exercise 20

Write a program that prints all the partitions of a given integer. A partition of n is a bag of integers a1, a2, ..., such that a1 + a1 + ... = n. (A "bag" is like a set, in the sense that the order doesn't matter, but it is also like a sequence in the sense that it may contain repeated elements.)

Input

The only input line has an integer n.

Output

Print a bag of integers.

Example

Input

4

Output

4

3 1

2 2

2 1 1

1 1 1 1

Solution

To solve this task, a recursive method will be used, the function calls itself. For each function call, a stack is used to store the numbers to print them later, 'remainder' is responsible for printing the complete stack when the number has been reached, otherwise it will call the function again to complete it, ' limit 'is in charge of adding numbers to the stack has control of the cycle, each time it ends it returns the recursion to the previous call.

The code for the solution in Python written is shown below.

```
def partition(n, limit, P):
   while limit > 0:
      P.append(limit)
      remainder = n - limit
      if remainder > limit:
         partition(remainder, limit, P)
      else:
         if remainder == 0:
            for n in P:
               print(n,end=' ')
            print()
         else:
            partition(remainder, remainder, P)
      del P[-1]
      limit -= 1

N = int(input())

partition(N, N, [])
```

Desktop Testing

Doing desktop testing can be confusing at first because of the use of recursion, but the more you analyze it, the better the understanding. It is observed that every ‘reaminder’ is 0, the stack is printed. Color is added to each beginning and end of each method for a better compression of the recursion, the end of the method arrives when 'limit' equals 0.

n	limit	P	reaminder	print
Method start n = 4 limit = 4 P = []				
4	4	[]		
		[4]	0	4
	3	[]		
		[3]	1	
Method start n = 1 limit = 1 P = [3]				
1	1	[3]		
		[3, 1]	0	3 1
	0	[3]		
End of method n = 1 limit = 0 P = [3]				
4	2	[]		
		[2]	2	
Method start n = 2 limit = 2 P = [2]				
2	2	[2]		
		[2, 2]	0	2 2
	1	[2]		
		[2, 1]	1	
Method start n = 1 limit = 1 P = [2, 1]				
1	1	[2, 1]		
		[2, 1, 1]	0	2 1 1
	0	[2, 1]		
End of method n = 1 limit = 0 P = [2, 1]				
2	1	[2, 1]		
	0	[2]		

End of method n = 2 limit = 0 P = [2]				
4	1	[]		
		[1]	3	
Method start n = 3 limit = 1 P = [1]				
3	1	[1]		
		[1, 1]	2	
Method start n = 2 limit = 1 P = [1, 1]				
2	1	[1, 1]		
		[1, 1, 1]	1	
Method start n = 1 limit = 1 P = [1, 1, 1]				
1	1	[1, 1, 1]		
		[1, 1, 1, 1]	0	1 1 1 1
	0	[1, 1, 1]		
End of method n = 1 limit = 0 P = [1, 1, 1]				
2	0	[1, 1]		
End of method n = 2 limit = 0 P = [1, 1]				
3	0	[1]		
End of method n = 3 limit = 0 P = [1]				
4	0	[]		
End of method n = 4 limit = 0 P = []				

Exercise 21

There are n apples with known weights. Your task is to divide the apples into two groups so that the difference between the weights of the groups is minimal.

Input

The first input line has an integer n, the number of apples.

The next line has n integers, the weight of each apple.

Output

Print one integer: the minimum difference between the weights of the groups.

Example

Input

6

Output

4 1 5 2 5 2

Solution

To solve this task, use the ‘itertools’ library using the ‘combinations’ method. The first for controls the length of each set, while the second goes through each possible combination looking for the smallest difference between the two sets. First ‘w1’ calculates the sum of the set and ‘w2’ subtracts the sum of the weights, if the difference of both is less than ‘ans’, it is substituted, it continues until the smallest possible difference is found.

The code for the solution written in Python is shown below.

```
from itertools import combinations
n = int(input())
weights = [int(i) for i in input().split()]
ans = sum(weights)
for i in range(1,n):
    for comb in combinations(weights,i):
        w1 = sum(comb)
        w2 = sum(weights)-sum(comb)
        if abs(w2-w1)<ans:
            ans = abs(w2-w1)
print(ans)
```

Desktop Testing

Doing desktop testing, it is observed how each set of combinations is iterated, saving in 'w1' and 'w2', in each iteration it is sought that the difference is minimal.

```
n = 4

weights = [2, 3, 6, 4]
```

i	comp	w1	w2	ans
				15
1	(2,)	2	13	11
	(3,)	3	12	9
	(6,)	6	9	3
	(4,)	4	11	
2	(2, 3)	5	10	
	(2, 6)	8	7	1
	(2, 4)	6	9	
	(3, 6)	9	6	
	(3, 4)	7	8	
	(6, 4)	10	5	
3	(2, 3, 6)	11	4	
	(2, 3, 4)	9	6	
	(2, 6, 4)	12	3	
	(3, 6, 4)	13	2	

Exercise 22

Your task is to place eight queens on a chessboard so that no two queens are attacking each other. As an additional challenge, each square is either free or reserved, and you can only place queens on the free squares. However, the reserved squares do not prevent queens from attacking each other.

How many possible ways are there to place the queens?

Input

The input has eight lines, and each of them has eight characters. Each square is either free (.) or reserved (*).

Output

Print one integer: the number of ways you can place the queens.

Example

Input

```
........
...*....
........
........
..*.....
........
.......*
........
```

Output

```
68
```

Solution

To solve this task, first the matrix where '.' Is replaced by 0 and '*' by 1. Next, the variables column 'col' with length 8, the diagonals 'diag1' and 'diag2' are started with length 15, and 'count' at 0. A recursive method is used to traverse the board, when 'n' reaches 8 it means that a move can be made so 'count' increases. The loop is used to cycle through each 'reserved' position as long as it is not reserved and both the column and the diagonals remain True, the values are changed to ensure that two queens do not touch.

The code for the solution written in Python is shown below.

```
reserved = [[0]*8 for _ in range(8)]
for i in range(8):
    row = list(str(input()))
    for j in range(8):
        if row[j] == '*':
            reserved[i][j] = 1

col = [True] * 8
diag1 = [True] * 15
diag2 = [True] * 15
count = 0
def chess(n):
   global count
   if n == 8:
      count += 1
      return
   for m in range(8):
      if reserved[n][m] == 0 and col[m] and diag1[m+n] and
diag2[7-n+m]:
         col[m] = diag1[m+n] = diag2[7-n+m] = False
         chess(n+1)
         col[m] = diag1[m+n] = diag2[7-n+m] = True

chess(0)
print(count)
```

Desktop Testing

Doing a desktop test of this algorithm is too long, so the reader is invited to debug the code to check its effectiveness.

Exercise 23

Write a program that reads a list of words from the standard input, with one word per line, and prints any one of the shortest words among those that appear least often.

Input

First line contain a integer n, number of words.

Each line contains one word.

Output

Print any one of the shortest words among those that appear least often.

Example

Input

4
water
sand
water
beach

Output

sand

Solution

To solve this task, each of the lines is read and stored in a dictionary to count the frequency. The variable ‘min_v’ is used to save the lowest frequency and ‘min_len_word’ to save the word of shortest length. By containing the value 'None' they start with the first values of the dictionary, otherwise if the value is less than 'min_v' the word and value are replaced, finally if the length is the same but the length of the word is less than 'min_len_word', it is replaced.

The code for the solution written in Python is shown below.

```
D = {}
n = int(input())
for w in range(n):
    w = input().strip()
    if w in D:
        D[w] += 1
    else:
        D[w] = 1

min_v = None
min_len_word = None
for k,v in D.items():
    if min_v == None:
        min_v = v
        min_len_word = k
    elif v < min_v:
        min_v = v
        min_len_word = k
    elif v == min_v and len(k) < len(min_len_word):
        min_len_word = k

print(min_len_word)
```

Desktop Testing

Doing a desktop testing, it is observed that despite 'sand' and 'beach' have the same frequency, the shortest word is 'sand'.

D	k	v	min_v	min_len_word
{'water': 2, 'sand': 1, 'beach': 1}			None	None
	water	2	2	water
	sand	1	1	sand
	beach	1		

```
The final result is: sand
```

Exercise 24

Write a program that reads a string of digits S as a command-line argument, and inserts plus (+) and minus signs (-) in the sequence of digits so as to create an expression that evaluates to zero. The program should output a resulting expression, if one such expression exists, or "None" if no such expressions exist.

Input

The only input line has an string of digits S.

Output

Print the resulting expression, if one such expression exists, or "None" if no such expressions exist.

Example

Input

132

Output

+1-3+2

Solution

To solve this task, a recursive method is used to traverse the string to find the appropriate sign. It has conditions to be able to exit the method for when the recursion occurs. A cycle is started where 'front', 'back' and 'target' are extracted, in this way the method is called recursively, returning the part of the string with the corresponding sign.

The code for the solution written in Python is shown below.

```
def solve(s, n):
    v = int(s)
    if v == n:
        return '+' + s

    if v == -n:
        return '-' + s

    if v < abs(n):
        return None

    for i in range(1,len(s)):
        front = s[0:i]
        back = s[i:]
        target = int(front)
        back_solution = solve(back,n-target)
        if back_solution != None:
            return '+' + front + back_solution
        back_solution = solve(back,n+target)
        if back_solution != None:
            return '-' + front + back_solution
    return None

print(solve(input(),0))
```

Desktop Testing

When doing the desktop test it is shown that three conditions must be met between 'v' and 'n', in the same way in 'back_solution' it contains parts of the chain that are dragged until reaching the main method, printing the complete result.

s	n	i	front	back	target	back_solution
Method start s = '132' n = 0						
'132'	0	1	'1'	'32'	1	
Method start s = '32' n = -1						
'32'	-1	1	'3'	'2'	3	
Method start s = '2' n = -4						
'2'	-4					None
End of method s = '2' n = -4						
Method start s = '2' n = 2						
'2'	2					None
End of method s = '2' n = 2						
						'+2'
End of method s = '32' n = -1						
						'-3+2'
End of method s = '132' n = 0						
						'+1-3+2'

Here ends the first edition of this book, if you have any questions or corrections contact me at stinktoignorance5@gmail.com.